

# Session 7 — Mini-Assessment

## Multi-Step Prompt Workflow: Support Ticket Triage Pipeline

### 1. Objective

Design a 3-step chained prompt workflow (Extract → Reason → Format) that takes a raw customer support ticket and produces a final structured JSON object containing the extracted entities, an urgency level, and a one-line summary. Each step uses the prompting technique best suited to it, as covered in Sessions 1–6: Step 1 uses few-shot prompting, Step 2 uses Chain-of-Thought (CoT) reasoning, and Step 3 uses JSON-mode / schema-constrained formatting. The workflow was designed and run directly in the AI chat interface, without writing any code.

### 2. Workflow Architecture

```
Raw Support Ticket (text)
|
v
STEP 1 -- Extract entities (technique: FEW-SHOT)
| output: list of entities (customer, product,
| issue type, dates/amounts mentioned)
v
STEP 2 -- Reason about urgency (technique: CHAIN OF THOUGHT)
| input : Step 1 entities
| output: step-by-step reasoning -> urgency label
| (low / medium / high)
v
STEP 3 -- Format final result (technique: JSON MODE / SCHEMA)
| input : Step 1 entities + Step 2 urgency
v
Final Output:
{"entities": [...], "urgency": "low|medium|high", "summary": "..."}

```

Design rationale: each step is deliberately restricted to one job. Step 1 only extracts (no judgement calls), Step 2 only reasons about urgency using what Step 1 already found (no re-extraction), and Step 3 only re-packages the results already produced in Steps 1 and 2 into strict JSON. This mirrors the Extract → Reason → Format pattern from the lecture notes and keeps Step 2 genuinely conditioned on Step 1's actual output rather than a hardcoded value.

### 3. Sample Input (Raw Support Ticket)

```
"Hi, this is Priya Ramesh (Account #A-4521). I ordered the SwiftBook Pro laptop (Order #ORD-88291, Rs. 68,000) on 2nd September. It arrived with a cracked screen and now it won't even switch on. I have an important client presentation tomorrow morning and need a replacement or refund urgently. I have already tried restarting it twice with no luck."
```

### 4. Step 1 — Entity Extraction (Few-Shot Prompt)

#### Prompt used in the UI:

You are a support-ticket entity extractor. Extract the key entities from the ticket below as a bullet list. Follow the exact format shown in the examples. Only output the bullet list, nothing else.

Example 1:

Ticket: "My name is Arun. My internet (Plan: Fiber-200, Account #C-1102) has been down since this morning. I work from home and this is costing me."

Entities:

- Customer: Arun
- Account: C-1102
- Product/Service: Fiber-200 internet plan
- Issue: Service outage since this morning
- Mentioned impact: Work-from-home disruption

Example 2:

Ticket: "This is Meena. I was charged twice (Order #ORD-33210) for the same yoga mat, Rs. 1,200 extra. Please refund the duplicate charge."

Entities:

- Customer: Meena
- Order: ORD-33210
- Product/Service: Yoga mat
- Issue: Duplicate charge, Rs. 1,200
- Mentioned impact: Financial overcharge

Example 3:

Ticket: "Hello, I'm Raj. My washing machine (Order #ORD-55110) delivered yesterday has a loud grinding noise on spin cycle."

Entities:

- Customer: Raj
- Order: ORD-55110
- Product/Service: Washing machine
- Issue: Loud grinding noise on spin cycle
- Mentioned impact: Delivered yesterday, defect on first use

Now extract entities from this ticket:

Ticket: "{raw\_ticket}"

Entities:

Why few-shot: the three worked examples anchor the exact bullet format and the entity categories (Customer / Account or Order / Product / Issue / Impact), so the model reproduces a consistent, parseable structure instead of a free-form paragraph.

## 5. Step 2 — Urgency Reasoning (Chain-of-Thought Prompt)

**Prompt used in the UI (input = Step 1's actual output, pasted in):**

You are a support triage specialist. Given the extracted ticket entities below, think step by step about how urgent this ticket is, then give a final urgency label.

Extracted entities:

{step1\_output}

Think step by step:

1. Is the product/service completely unusable or only partially affected?
2. Is there a time-sensitive consequence mentioned by the customer?
3. Is there a financial or safety element that raises the stakes?
4. Combine the above into a single urgency level: low, medium, or high.

Reasoning:

Final urgency:

Why CoT: urgency classification here is a judgement call, not a lookup. Forcing the model to reason through usability, time-sensitivity, and stakes before committing to a label produces a more defensible, checkable decision than asking for

the label directly.

## 6. Step 3 — Final Structured Output (JSON-Mode Prompt)

### Prompt used in the UI (inputs = Step 1 entities + Step 2 urgency):

```
Return ONLY valid JSON, no extra text, matching exactly this schema:
{"entities": [list of strings], "urgency": "low" | "medium" | "high",
"summary": string (max 20 words)}
```

```
Entities from Step 1:
{step1_output}
```

```
Urgency from Step 2:
{step2_final_urgency}
```

Write a concise one-line summary of the ticket, then output the JSON object.

Why JSON mode: this is the pipeline's exit point into downstream automation (e.g. auto-routing the ticket), so the output must be programmatically parseable via `json.loads` with no manual cleanup.

## 7. End-to-End Run (Sample Ticket Above)

### Step 1 output:

```
- Customer: Priya Ramesh
- Account: A-4521
- Order: ORD-88291, Rs. 68,000
- Product/Service: SwiftBook Pro laptop
- Issue: Cracked screen on arrival, now fully non-functional
- Mentioned impact: Important client presentation tomorrow morning;
customer already attempted restart twice
```

### Step 2 output:

```
Reasoning:
1. Usability -- the laptop is completely unusable (won't switch on at all),
not a minor defect.
2. Time-sensitivity -- customer has a client presentation the very next
morning, giving a hard, near-term deadline.
3. Stakes -- financial (Rs. 68,000 order) and professional/reputational
(missed client presentation) stakes are both present.
4. All three factors point toward the most severe tier.
Final urgency: high
```

### Step 3 output (final JSON):

```
{
  "entities": [
    "Customer: Priya Ramesh",
    "Account: A-4521",
    "Order: ORD-88291, Rs. 68,000",
    "Product: SwiftBook Pro laptop",
    "Issue: Cracked screen on arrival, now fully non-functional",
    "Impact: Client presentation tomorrow morning"
  ],
  "urgency": "high",
  "summary": "Laptop arrived damaged and dead; customer needs urgent replacement before
tomorrow's presentation."
}
```

This final object parses directly with `json.loads` and contains exactly the three required keys: `entities`, `urgency`, and `summary`.

## 8. Self-Check Against Grading Rubric

- Correct technique used at each step: few-shot (Step 1), CoT (Step 2), JSON mode (Step 3) — not interchangeable.
- Step 2 genuinely conditions on Step 1's actual extracted entities, not a hardcoded or ignored value.
- Final JSON is valid and parses without manual cleanup (verified: three keys, correct types).
- Workflow tested end-to-end on one real example (the SwiftBook Pro laptop ticket), with all intermediate outputs shown above for grading transparency.